FIAP – Faculdade de Informática e Administração Paulista
Engenharia de Software – 2º ano
Challenge 2026 – TOTVS

TOTVS Inteligência Conversacional

Plataforma de análise automatizada de reuniões comerciais com inteligência artificial

Domain Driven Design – Java – Sprint 3
Camadas Model, Service, Connection e DAO (JDBC) sobre o Oracle Database da solução

Grupo Company 404
Bruno Gama Masseiras – RM 562526
Kauai Rosa de Assis Rocha – RM 563256
Lucas Gilmar Pedro da Silva – RM 566231
Leandro Afonso Silva Santos Junior – RM 561344
Murilo da Silva Basto – RM 565445

São Paulo
Setembro de 2026

Grupo Company 404
Bruno Gama Masseiras – RM: 562526
Kauai Rosa de Assis Rocha – RM: 563256
Lucas Gilmar Pedro da Silva – RM: 566231
Leandro Afonso Silva Santos Junior – RM: 561344
Murilo da Silva Basto – RM: 565445
Domain Driven Design – Java – Challenge 2026 TOTVS – Sprint 3 – 2º ano

Projeto Java do TOTVS Inteligência Conversacional – Sprint 3

Sumário

1. Objetivo e escopo do projeto
2. Principais funcionalidades da solução
3. Protótipo: telas do sistema e como interagir
4. Modelo do banco de dados
5. Diagrama de classes atualizado
6. Projeto Java
6.1 Estrutura de pacotes e camada Model
6.2 Métodos de relevância (regras de negócio)
6.3 Camada de conexão
6.4 Camada DAO com CRUD
6.5 Classes de teste e evidências de execução
6.6 Como executar
7. Documentos de referência

1. Objetivo e escopo do projeto

O **TOTVS Inteligência Conversacional** transforma transcrições de reuniões comerciais e de Customer Success da TOTVS em inteligência acionável: a partir do texto da reunião, a solução identifica produtos TOTVS em uso, concorrentes citados, oportunidades de upsell, cross-sell e novo produto, riscos de churn, insatisfação e perda de confiança, o sentimento geral da conversa, a persona e o orçamento do interlocutor, e sugere a próxima ação comercial. Os resultados alimentam alertas priorizados e um dashboard executivo, na web e no aplicativo Android.

Nas Sprints 1 e 2, o projeto Java demonstrou os conceitos de orientação a objetos (encapsulamento, herança, polimorfismo e composição) com um analisador por palavras-chave lendo o arquivo real da TOTVS. Nesta Sprint 3, o projeto evolui para a **arquitetura em camadas usada em sala**: a camada **Model** representa as entidades do domínio (transcrição, análise, insight, oportunidade, risco, produto, alerta, usuário e feedback) com os mesmos atributos das tabelas Oracle; a camada **Service** concentra as regras de negócio; a camada de **Connection** abre a conexão JDBC com o Oracle da FIAP (usuário e senha no código, conforme pedido); e a camada **DAO** implementa o CRUD completo de cada entidade. Duas classes de teste exercitam, respectivamente, as regras de negócio (sem banco) e o CRUD contra o banco real.

O escopo é o mesmo domínio da solução em produção (backend Spring Boot, frontend Next.js, LeanIA em Python e Oracle 19c). O projeto Java desta entrega é independente de frameworks e usa exatamente o schema entregue em Database Design na Sprint 3, o que permite ao professor executar o CRUD no schema do grupo ou em qualquer schema criado com o script DDL.

2. Principais funcionalidades da solução

Funcionalidade	Descrição	No projeto Java
Ingestão de transcrições	Formulário web, API REST, importação em lote dos arquivos JSON e CSV da TOTVS e transcrição de áudio.	Transcricao, TranscricaoDAO, TranscricaoService
Análise da reunião (LeanIA)	Sentimento, produtos e concorrentes, oportunidades com score, riscos com severidade, persona, budget, confiança no vendedor, resumo e próxima ação.	AnalisadorTranscricao, Analise, Insight, Oportunidade, Risco, Produto
Alertas acionáveis	Alertas por risco, concorrente e oportunidade alta, com prioridade e ciclo NOVO → VISTO → RESOLVIDO.	GeradorAlertas, Alerta, AlertaDAO
Priorização de contas	Ordena os clientes por urgência de atuação combinando riscos, oportunidades, NPS e recência da reunião.	PriorizadorContas
Qualidade da conversa	Participação de vendedor e cliente, perguntas, turnos e escuta ativa a partir dos locutores da transcrição.	CalculadoraIndicadores
Feedback adaptativo	O usuário avalia a análise (útil ou ajuste) e o histórico entra como contexto das próximas análises.	Feedback, FeedbackDAO
Segurança e perfis	Usuários com perfil ADMIN ou USER; o usuário comum só acessa o que criou.	Usuario, UsuarioDAO
Dashboard executivo	Métricas consolidadas, gráficos de sentimento, riscos e oportunidades, reuniões e alertas recentes (web e Android).	consultas dos DAOs (listar, contar, por status)

3. Protótipo: telas do sistema e como interagir

As telas são capturas da aplicação web em produção, com os dados reais da base de 623 reuniões analisadas. Para cada tela, o texto explica como o usuário interage e qual classe do projeto Java realiza a operação correspondente.

3.1 Tela de login

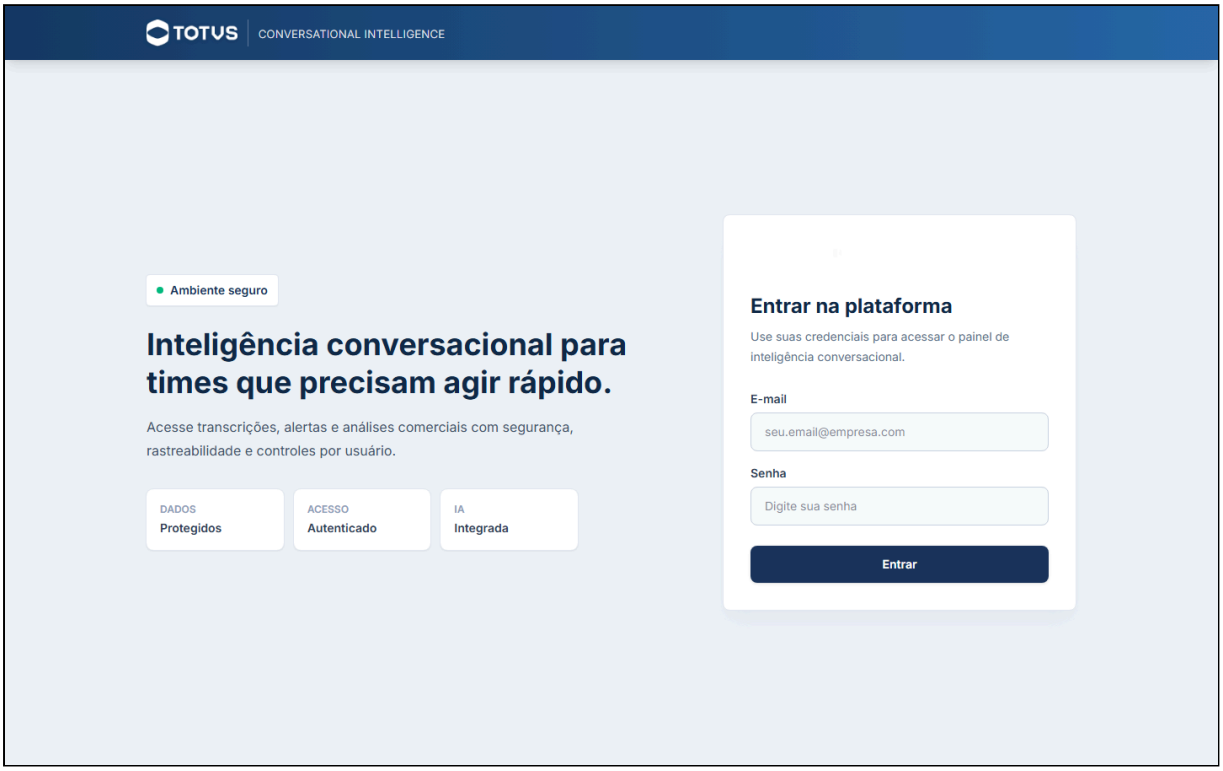


Figura 1 – Tela de login.

Informe o e-mail e a senha cadastrados e clique em **Entrar**. Após a autenticação, o sistema abre o dashboard; a sessão é renovada automaticamente e o botão **Sair** encerra o acesso. No projeto Java, o cadastro e a consulta de usuários são feitos por UsuarioDAO e a regra de acesso por Usuario.podeAcessar.

3.2 Dashboard executivo

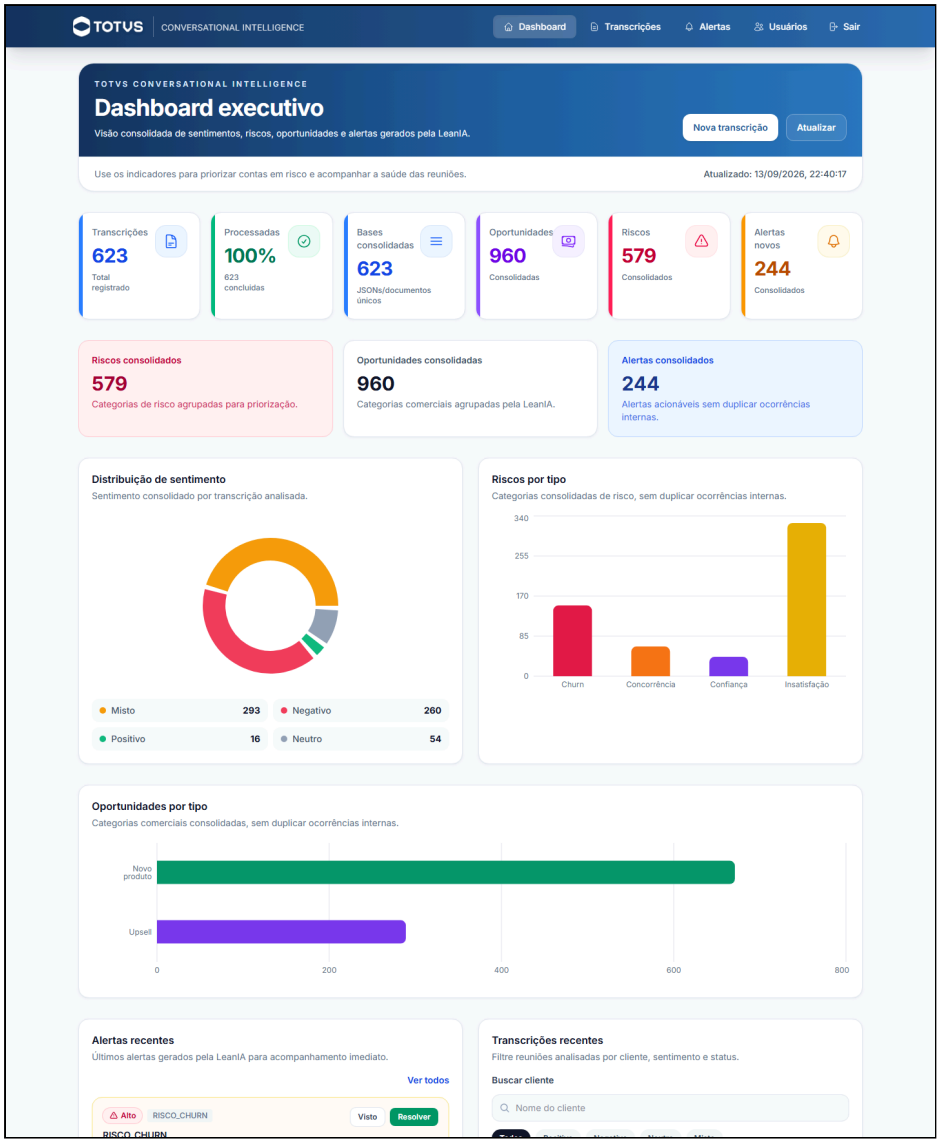


Figura 2 – Dashboard executivo.

Primeira tela após o login. Os seis cartões mostram os totais (transcrições, processadas, oportunidades, riscos e alertas novos); os gráficos detalham sentimento, riscos por tipo e oportunidades por tipo. Use **Nova transcrição** para cadastrar uma reunião, **Atualizar** para recarregar os dados, e os blocos de alertas e transcrições recentes para ir direto ao registro. Os totais equivalem às consultas listar e contar dos DAOs.

3.3 Gestão de transcrições

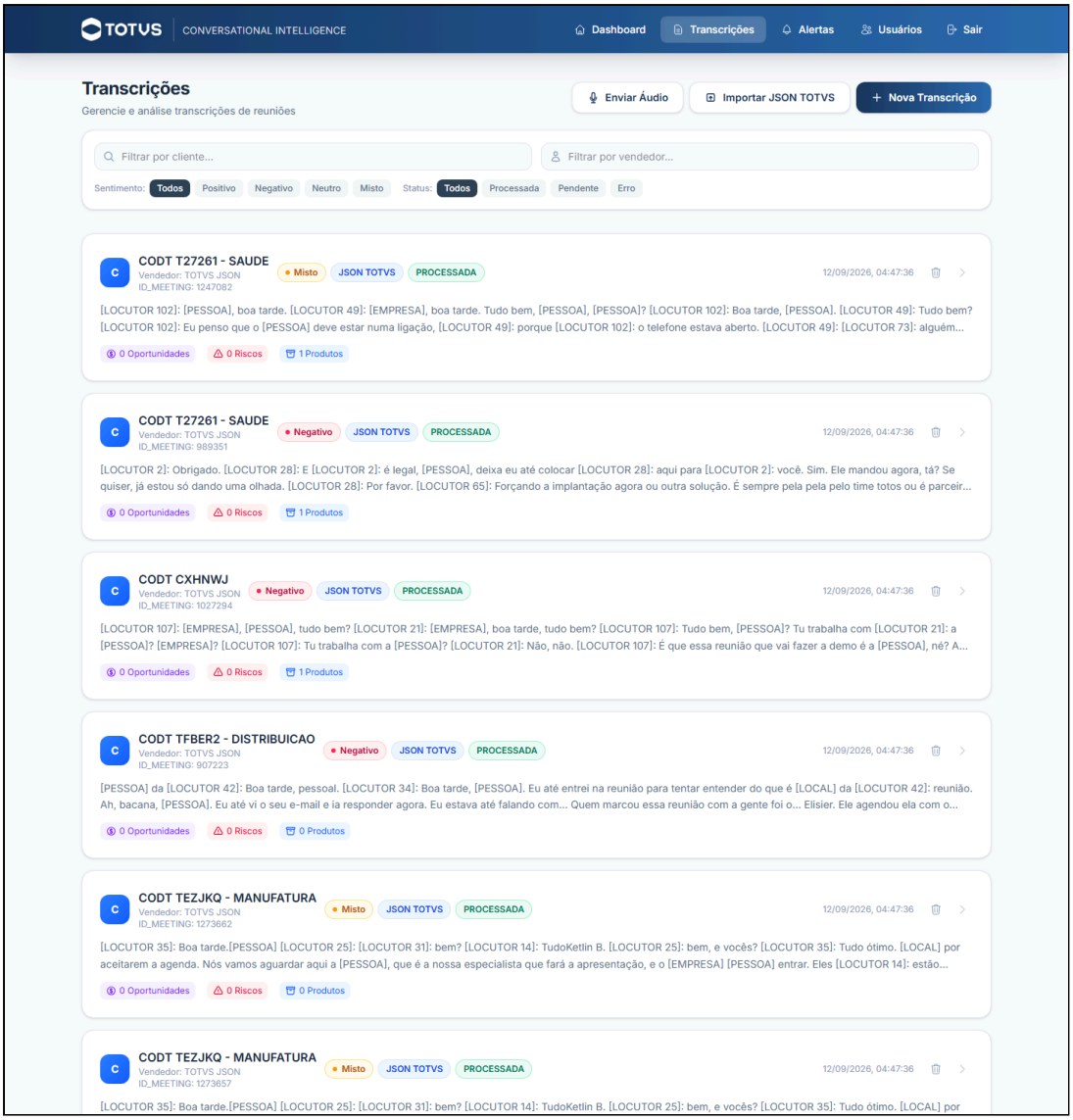


Figura 3 – Gestão de transcrições.

Lista as reuniões com cliente, vendedor, origem, status e sentimento. Filtre por cliente ou vendedor no topo e por sentimento e status nos botões. **Nova transcrição** abre o formulário de texto, **Importar JSON TOTVS** envia o arquivo da TOTVS e **Enviar áudio** transcreve um arquivo de áudio. Clique no cartão para abrir o detalhe; o ícone de lixeira exclui a reunião (TranscricaoDAO.excluir, com exclusão em cascata da análise e dos alertas).

3.4 Detalhe da análise

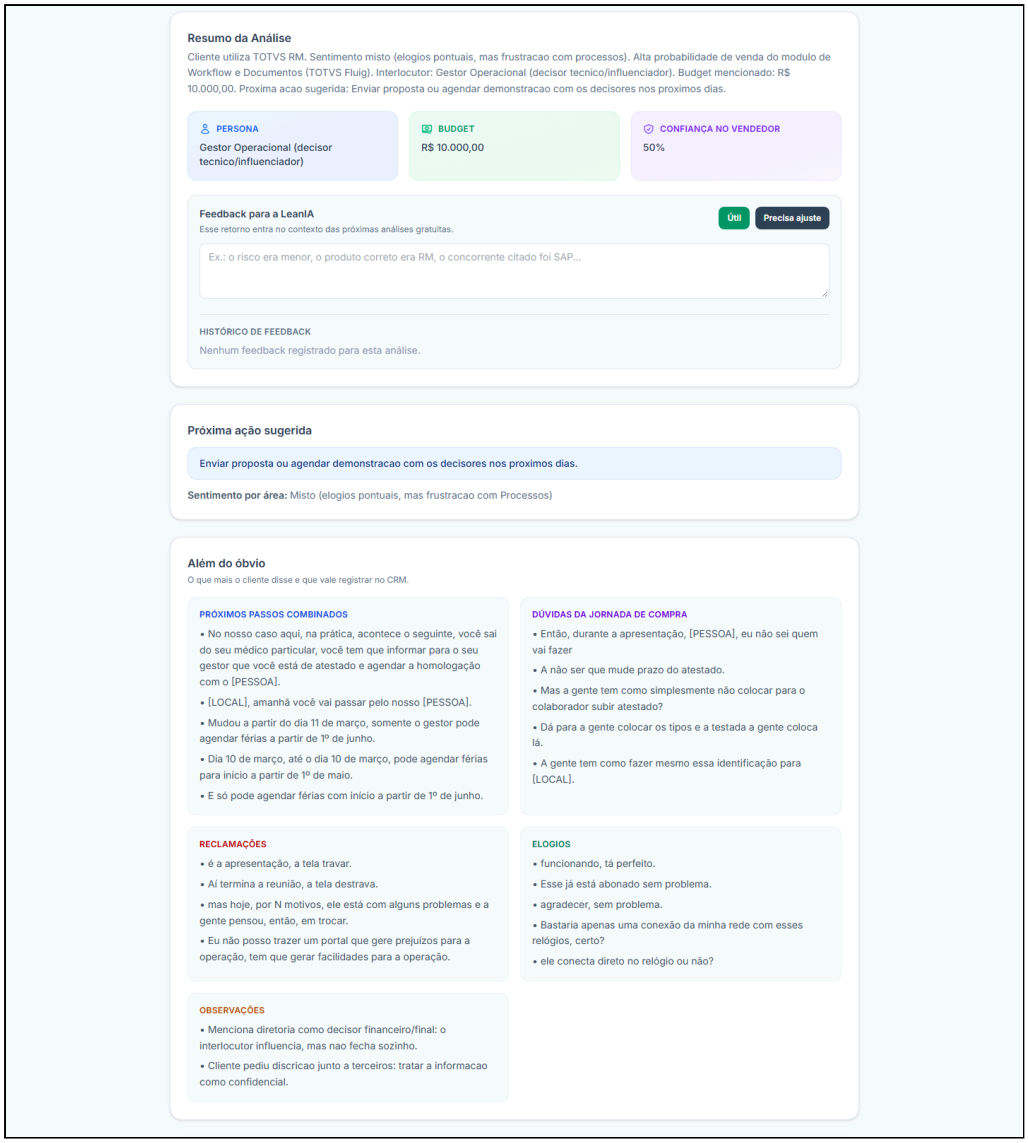


Figura 4 – Detalhe da análise.

Mostra a transcrição original e, abaixo, o resumo da análise, persona, budget e confiança no vendedor, a próxima ação sugerida e os insights além do óbvio. Em **Feedback para a LeanIA**, clique em **Útil** ou **Precisa ajuste** e, se quiser, descreva a correção; o histórico aparece logo abaixo (FeedbackDAO). Os botões **CSV** e **PDF** exportam a análise; **Reprocessar** refaz a análise (TranscricaoService.processar).

3.5 Central de alertas

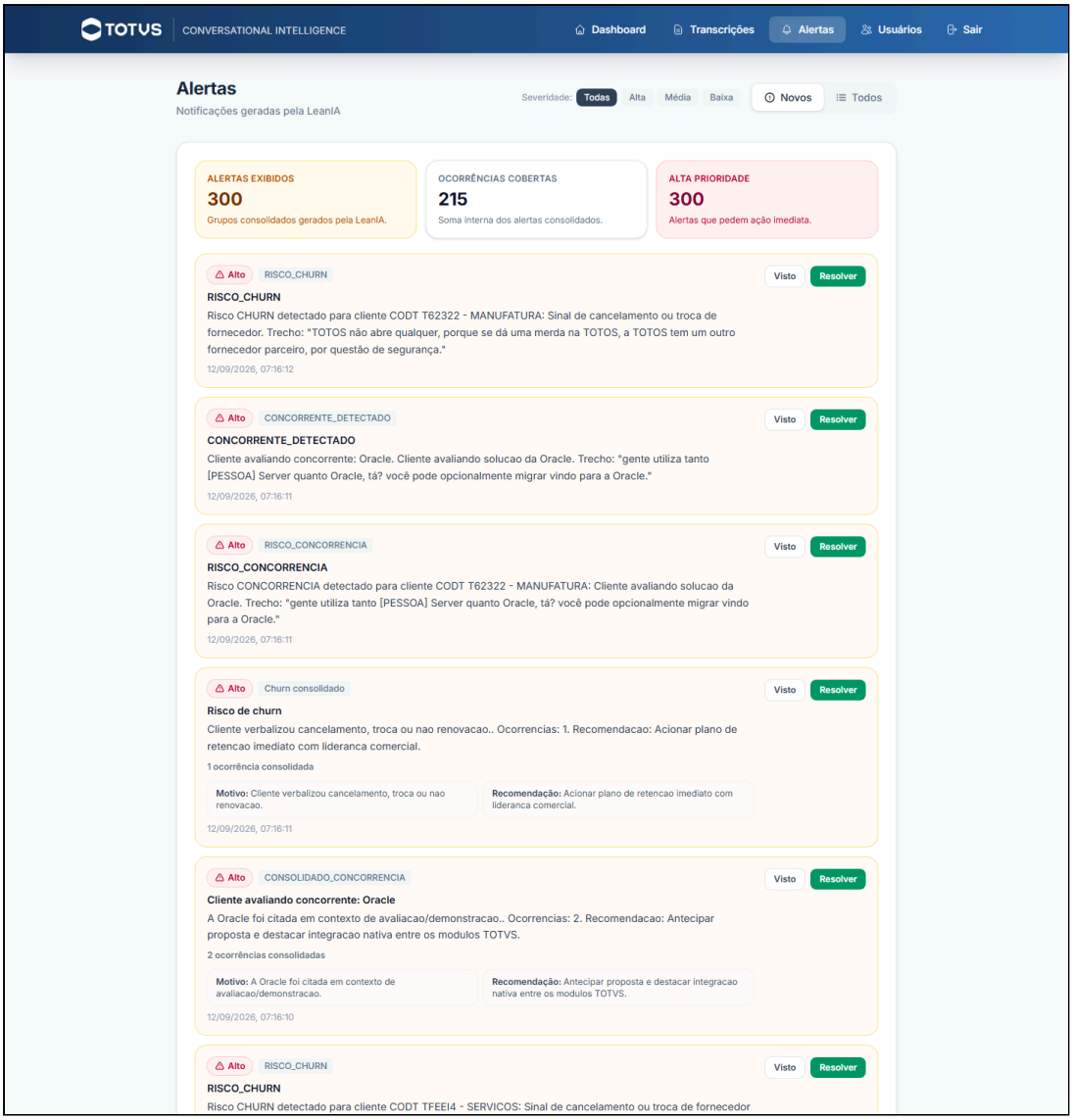


Figura 5 – Central de alertas.

Fila de alertas gerados pela análise, com severidade, tipo, cliente e o trecho que originou o alerta. Filtre por severidade e alterne entre **Novos** e **Todos**. Em cada cartão, **Visto** marca o alerta como visto e **Resolver** o encerra; a ordem das ações é obrigatória (Alerta.marcarVisto e Alerta.resolver, persistidos por AlertaDAO.atualizar).

4. Modelo do banco de dados

O projeto Java persiste os objetos no mesmo modelo relacional da solução, construído no Oracle SQL Developer Data Modeler e entregue em Database Design (Sprint 3): 20 tabelas, 24 chaves estrangeiras, sequence, colunas IDENTITY e triggers de histórico. As classes do Model correspondem às tabelas APP_USERS (Usuario), TRANSCRICOES (Transcricao), ANALISES (Analise), ANALISE_PRODUTOS (Produto), ANALISE_OPORTUNIDADES (Oportunidade), ANALISE_RISCOS (Risco), ALERTAS (Alerta) e ANALISE_FEEDBACKS (Feedback).

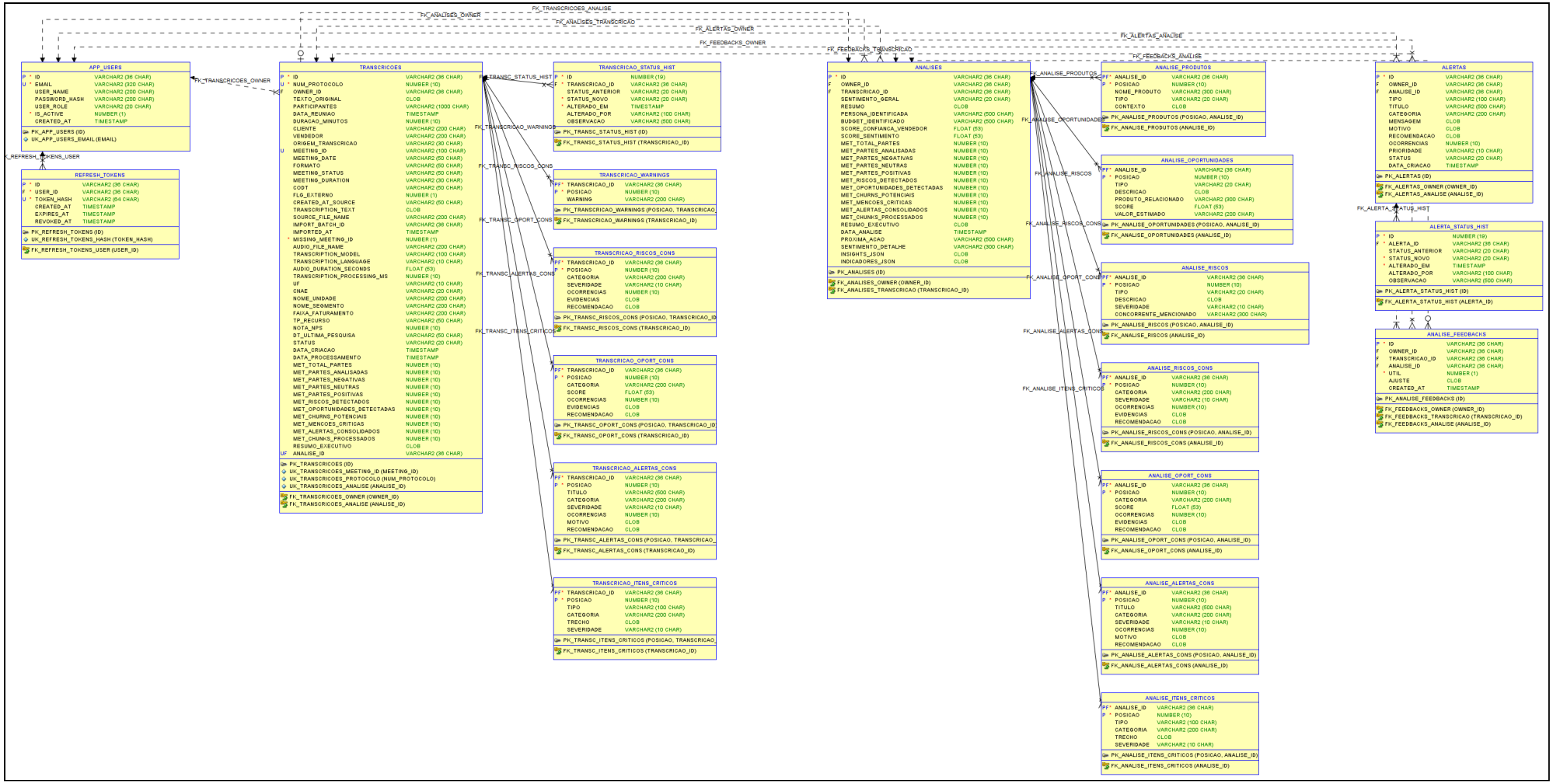


Figura 6 – Modelo relacional completo exportado do Oracle SQL Developer Data Modeler.

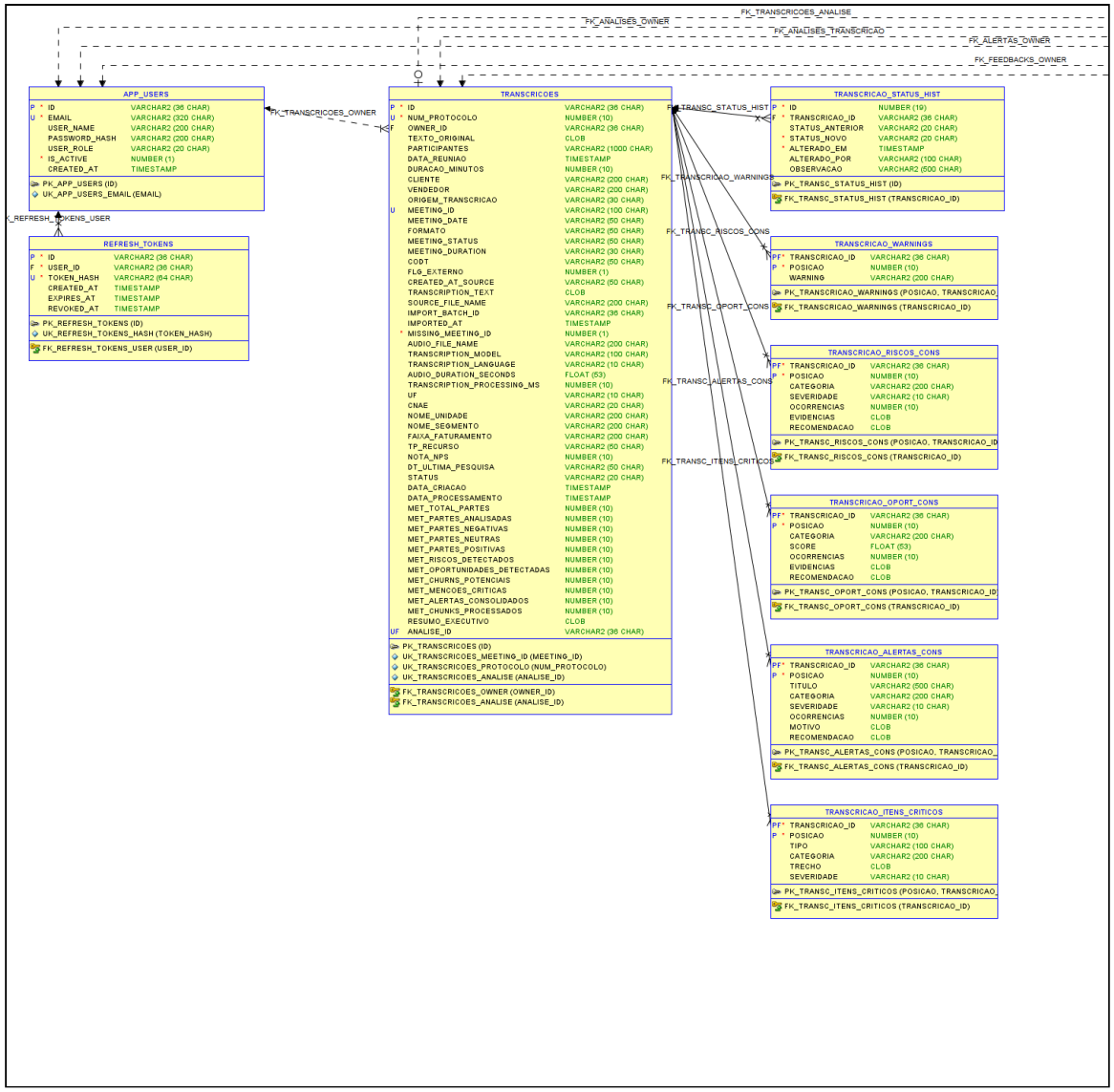


Figura 7 – Metade esquerda do modelo: usuários, transcrições e listas consolidadas.

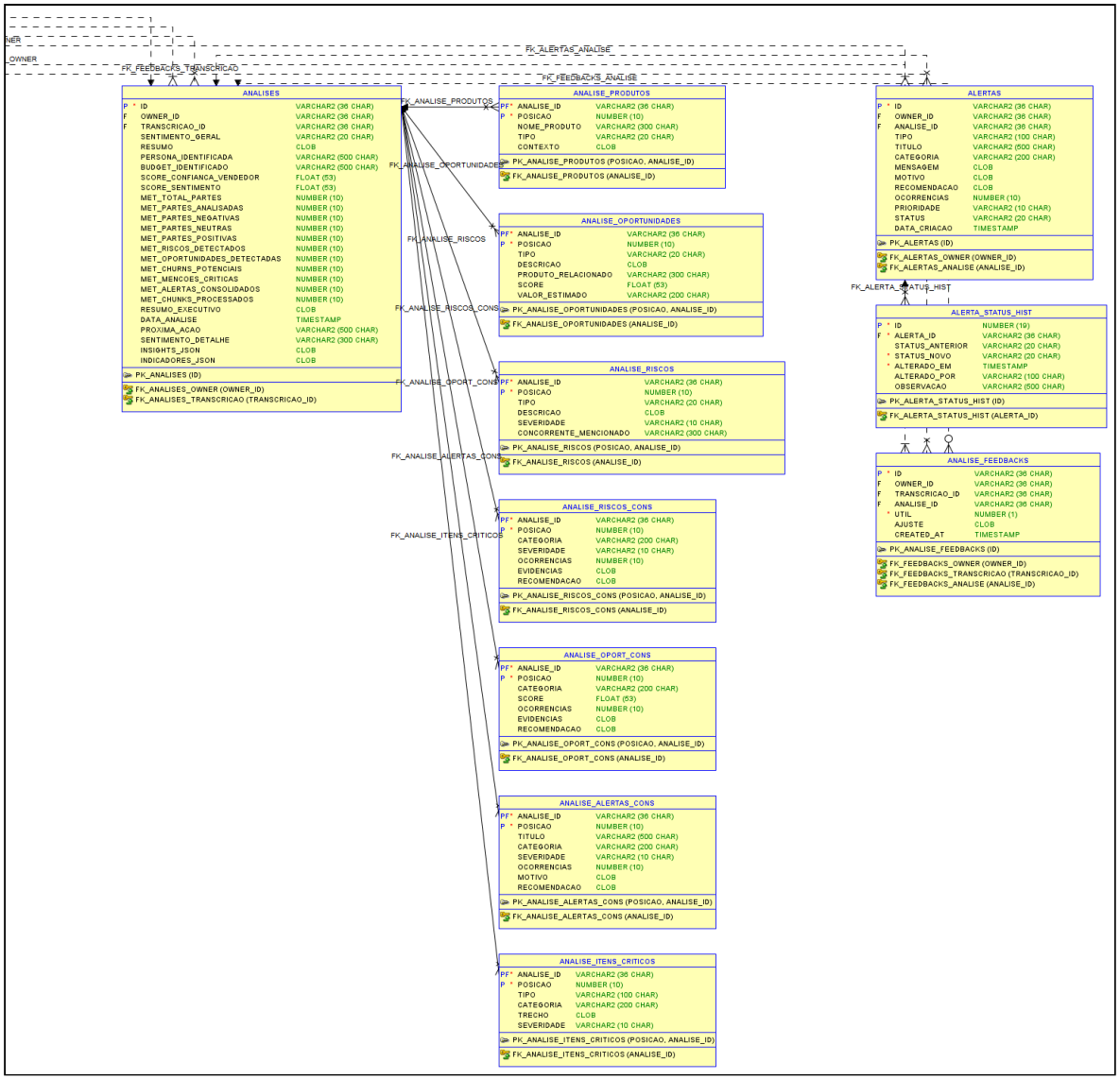


Figura 8 – Metade direita do modelo: análises, detalhes, alertas, histórico e feedbacks.

Classe Java	Tabela Oracle	Observações de mapeamento
Usuario	APP_USERS	ID UUID gerado no construtor; IS_ACTIVE 0/1 vira boolean; USER_ROLE vira o enum Perfil.
Transcricao	TRANSCRICOES	Colunas principais da reunião; STATUS vira StatusTranscricao; ANALISE_ID guarda a referência 1:1 para a análise atual.
Analise	ANALISES	Sentimento, resumo, persona, budget, scores, próxima ação; as listas ficam nas tabelas filhas.
Produto	ANALISE_PRODUTOS	Chave composta (POSICAO, ANALISE_ID); POSICAO preserva a ordem da lista.
Oportunidade / Risco	ANALISE_OPORTUNIDADES / ANALISE_RISCOS	Subclasses de Insight; AnaliseDAO grava e lê as duas listas na mesma transação.
Alerta	ALERTAS	PRIORIDADE vira Severidade e STATUS vira StatusAlerta; ON DELETE CASCADE a partir da análise.
Feedback	ANALISE_FEEDBACKS	UTIL 0/1 vira boolean; ANALISE_ID fica NULL se a análise for refeita (SET NULL).

5. Diagrama de classes atualizado

A Figura 9 apresenta as classes do projeto com atributos (visibilidade, nome e tipo) e métodos, organizadas nos pacotes model, service e dao/connection. Em relação às Sprints 1 e 2, a herança Insight ← Oportunidade / Risco e o polimorfismo da lista de insights foram mantidos; Transcricao, Analise, Alerta, Usuario e Feedback ganharam os atributos das tabelas Oracle e os métodos de regra de negócio; entraram a interface genérica DAO<T>, as cinco implementações DAO, a ConnectionFactory e os serviços.

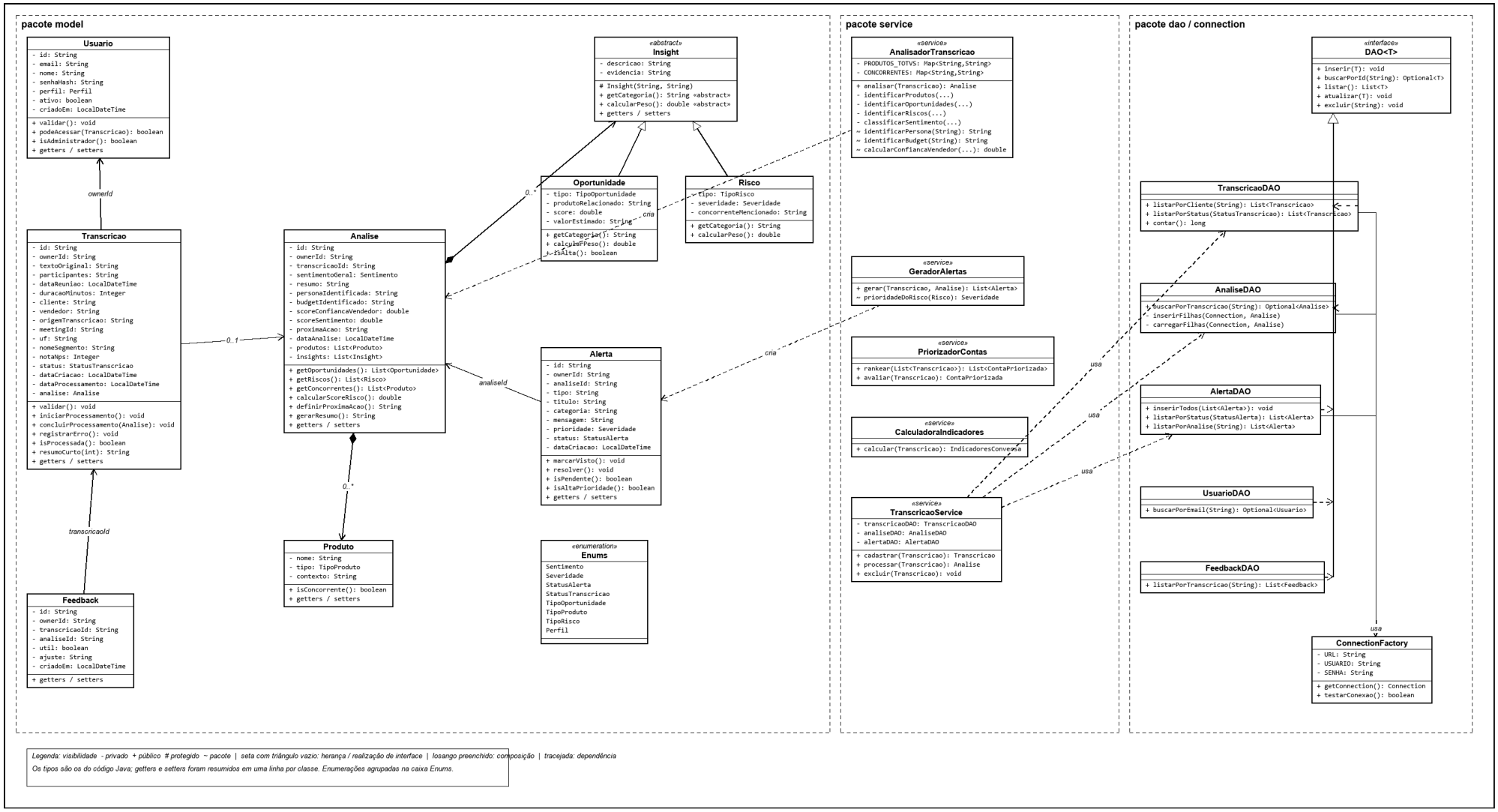


Figura 9 – Diagrama de classes do projeto Java (Sprint 3).

6. Projeto Java

6.1 Estrutura de pacotes e camada Model

```
com.tlean07.fiap.challenge
├── model        Transcricao, Analise, Insight (abstrata) ← Oportunidade / Risco, Produto, Alerta,
Usuario, Feedback
│                e as enumerações Sentimento, Severidade, StatusAlerta, StatusTranscricao,
TipoOportunidade, TipoProduto, TipoRisco, Perfil
├── service      AnalisadorTranscricao, GeradorAlertas, PriorizadorContas, CalculadoraIndicadores,
TranscricaoService
├── connection   ConnectionFactory
├── dao          DAO<T> (interface CRUD), TranscricaoDAO, AnaliseDAO, AlertaDAO, UsuarioDAO,
FeedbackDAO
├── exception    RegraNegocioException, DAOException
└── test         TesteRegrasNegocio (sem banco), TesteDAO (CRUD no Oracle)
```

A camada Model segue os padrões trabalhados em sala: atributos privados com getters e setters, construtor padrão e construtor com os campos principais, identificador UUID gerado no construtor (mesma estratégia do backend), toString para os relatórios, enumerações para os domínios fechados (status, sentimento, severidade, tipos) e exceção própria de regra de negócio. A classe abstrata Insight define os métodos abstratos getCategoria e calcularPeso; Oportunidade e Risco herdam dela e os sobrescrevem, e Analise guarda ambas em uma única List<Insight> (polimorfismo), com getOportunidades e getRiscos filtrando por tipo. Transcricao possui uma Analise (composição 1:1) e Analise possui listas de Produto e Insight.

6.2 Métodos de relevância (regras de negócio)

Classe	Método	Lógica implementada
AnalisadorTranscricao	`analisar(Transcricao)`	Normaliza o texto (acentos, caixa), separa sentenças por locutor, identifica produtos TOTVS e concorrentes (catálogo), classifica cada produto como em uso ou argumento de venda pelo contexto da frase, detecta oportunidades (intenção de compra 0,85; interesse 0,60) e riscos (churn, perda de confiança, concorrência em negociação, insatisfação com severidade por intensificadores), classifica o sentimento com léxico, negação (“não gostei”) e intensificadores, extrai persona e budget (R$, mil, por mês) com expressões regulares, calcula a confiança no vendedor e gera o resumo com a próxima ação.
GeradorAlertas	`gerar(Transcricao, Analise)`	Um alerta por tipo de risco (sem duplicar), alerta por concorrente citado e por oportunidade alta; churn e perda de confiança recebem prioridade ALTO; churn alto combinado com concorrente ou perda de confiança gera o alerta CHURN_IMINENTE no topo da lista.

PriorizadorContas	`rankear(List<Transcricao>)`	Score de urgência = soma polimórfica dos pesos dos insights (riscos somam, oportunidades subtraem) + penalidade por sentimento negativo + 3 pontos para NPS detrator, −1 para promotor, +1 para reunião nos últimos 7 dias; devolve as contas ordenadas com o motivo.
CalculadoraIndicadores	`calcular(Transcricao)`	Reconhece os turnos "[LOCUTOR N]:", conta palavras e perguntas por locutor, assume o locutor que mais fala como vendedor, calcula as participações e classifica a escuta ativa (equilibrado, falou mais, ouviu mais).
Transcricao	`validar()`, `iniciarProcessamento()`, `concluirProcessamento(Analise)`, `registrarErro()`	Validação (cliente obrigatório, texto mínimo, duração positiva, NPS 0 a 10) e máquina de estados PENDENTE → PROCESSANDO → PROCESSADA / ERRO, rejeitando transições inválidas com RegraNegocioException.
Alerta	`marcarVisto()`, `resolver()`	Ciclo NOVO → VISTO → RESOLVIDO; um alerta não pode ser resolvido sem ter sido visto.
Analise	`calcularScoreRisco()`, `definirProximaAcao()`, `gerarResumo()`	Score de risco da conta e escolha da próxima ação conforme churn, perda de confiança, oportunidade alta e budget.
Usuario	`validar()`, `podeAcessar(Transcricao)`	E-mail válido, nome mínimo, perfil obrigatório; ADMIN acessa tudo, USER só as próprias reuniões, inativo não acessa.

6.3 Camada de conexão

A classe `ConnectionFactory` (pacote connection) guarda a URL JDBC do Oracle da FIAP (`jdbc:oracle:thin:@//oracle.fiap.com.br:1521/orcl`), o usuário (RM561344) e a senha como constantes no código, conforme solicitado na entrega. O método `getConnection()` abre a conexão com `DriverManager` e desliga o autocommit, para que os DAOs controlem a transação; `testarConexao()` executa `SELECT 1 FROM DUAL`. Falhas de conexão viram `DAOException`, que encapsula a `SQLException` original.

6.4 Camada DAO com CRUD

A interface genérica `DAO<T>` define inserir, buscarPorId, listar, atualizar e excluir. Cada DAO usa `PreparedStatement` com parâmetros (sem concatenação de SQL), try-with-resources para fechar conexões e statements, commit explícito e mapeamento ResultSet → objeto.

DAO	Tabela(s)	Operações
TranscricaoDAO	TRANSCRICOES	inserir, buscarPorId, listar (100 mais recentes), listarPorCliente, listarPorStatus, atualizar, excluir, contar.

AnaliseDAO	ANALISES, ANALISE_PRODUTOS, ANALISE_OPORTUNIDADES, ANALISE_RISCOS	inserir e atualizar gravam a análise e as três listas na mesma transação (batch); buscarPorId e buscarPorTranscricao carregam as listas; excluir apaga a análise e o banco remove detalhes e alertas em cascata.
AlertaDAO	ALERTAS	inserir, inserirTodos (lote), buscarPorId, listar, listarPorStatus, listarPorAnalise, atualizar (status), excluir.
UsuarioDAO	APP_USERS	inserir (valida antes), buscarPorId, buscarPorEmail, listar, atualizar, excluir.
FeedbackDAO	ANALISE_FEEDBACKS	inserir, buscarPorId, listar, listarPorTranscricao, atualizar, excluir.

A classe `TranscricaoService` orquestra o fluxo completo com os DAOs: cadastrar (valida e insere), processar (PENDENTE → PROCESSANDO, analisa, grava a análise e os alertas, PROCESSADA; em caso de erro, ERRO) e excluir.

6.5 Classes de teste e evidências de execução

TesteRegrasNegocio instancia usuários, transcrições, análises e alertas e verifica os métodos de relevância sem banco de dados (29 verificações). Saída da execução:

```
=== TOTVS Inteligencia Conversacional - Teste das regras de negocio ===

-- Transcricao: validacao e ciclo de status
   [OK]     transcricao sem cliente e texto curto e rejeitada
   [OK]     status inicial PENDENTE
   [OK]     PENDENTE -> PROCESSANDO
   [OK]     nao pode iniciar duas vezes

-- AnalisadorTranscricao
   Analise{NEGATIVO | produtos=2 | oportunidades=0 | riscos=3 | scoreRisco=14,5}
   - RISCO INSATISFACAO: Insatisfacao relatada pelo cliente | Evidencia: "entendo a frustracao, vou
escalar hoje" | Severidade: BAIXO
   - RISCO CONCORRENCIA: Concorrente em negociacao com o cliente | Evidencia: "a senior ja nos
apresentou uma proposta e a diretoria esta pensando em trocar" | Severidade: MEDIO | Concorrente:
Senior
   - RISCO CHURN: Cliente verbalizou cancelamento ou troca de fornecedor | Evidencia: "se nao
resolver ate o fim do mes, vamos cancelar o contrato" | Severidade: ALTO
   [OK]     sentimento NEGATIVO na reuniao de churn
   [OK]     risco de CHURN detectado
   [OK]     concorrente Senior identificado
   [OK]     TOTVS RM identificado como produto atual
   [OK]     proxima acao e plano de retencao
   Analise{POSITIVO | produtos=2 | oportunidades=3 | riscos=0 | scoreRisco=0,0}
   - OPORTUNIDADE UPSELL: Cliente sinaliza intencao de compra: precisamos de uma proposta ate o fim
do mes | Evidencia: "precisamos de uma proposta ate o fim do mes" | Produto: TOTVS Fluig | Score:
85% | Valor: Nao estimado
   - OPORTUNIDADE UPSELL: Cliente sinaliza intencao de compra: o orcamento previsto para automacao e
de uns r$ 60 mil | Evidencia: "o orcamento previsto para automacao e de uns r$ 60 mil" | Produto:
TOTVS Fluig | Score: 85% | Valor: R$ 60 mil
   - OPORTUNIDADE UPSELL: Cliente demonstra interesse: gostei | Evidencia: "gostei" | Produto: TOTVS
Fluig | Score: 60% | Valor: Nao estimado
   [OK]     oportunidade alta para o Fluig
   [OK]     budget R$ 60.000,00 extraido
   [OK]     persona de decisor identificada
   [OK]     confianca no vendedor maior que 0,5
   Resumo: Cliente utiliza TOTVS Protheus. Sentimento positivo. Alta probabilidade de venda (TOTVS
Fluig). Alta probabilidade de venda (TOTVS Fluig). Possivel probabilidade de venda (TOTVS Fluig).
Interlocutor: Socio/Executivo (decisor final). Budget mencionado: R$ 60.000,00. Proxima acao
sugerida: Enviar proposta com o budget mencionado (R$ 60.000,00) nos proximos dias.
   [OK]     reuniao sem sinais fica NEUTRA
   [OK]     PROCESSANDO -> PROCESSADA com analise vinculada
```

```
-- GeradorAlertas e ciclo do alerta
   - [ALTO] CHURN_IMINENTE - Churn iminente (NOVO)
   - [BAIXO] RISCO_INSATISFACAO - RISCO_INSATISFACAO (NOVO)
   - [MEDIO] RISCO_CONCORRENCIA - RISCO_CONCORRENCIA (NOVO)
   - [ALTO] RISCO_CHURN - RISCO_CHURN (NOVO)
   - [MEDIO] CONCORRENTE_DETECTADO - CONCORRENTE_DETECTADO (NOVO)
   [OK]     alerta CHURN_IMINENTE gerado (churn + concorrente)
   [OK]     alerta de churn tem prioridade ALTO
   [OK]     resolver alerta NOVO direto e rejeitado
   [OK]     NOVO -> VISTO -> RESOLVIDO
   [OK]     oportunidade alta gera alerta OPORTUNIDADE_ALTA

-- PriorizadorContas
   Clinica Vida Plena           score  18,5  3 risco(s), NPS detrator (3), reuniao recente
   Colegio Nova Geracao         score   0,0  1 oportunidade(s)
   Metalurgica Andrade          score   0,0  3 oportunidade(s)
   [OK]     conta com churn e NPS detrator fica em primeiro
   [OK]     conta com oportunidade fica por ultimo

-- CalculadoraIndicadores
   Indicadores{locutores=3, turnos=5, palavras=59, vendedor=41%, cliente=59%, perguntasVendedor=0,
perguntasCliente=0, escuta=Equilibrado}
   [OK]     tres locutores reconhecidos
   [OK]     participacoes somam 100%

-- Usuario: controle de acesso
   [OK]     vendedor acessa a propria reuniao
   [OK]     vendedor nao acessa reuniao de outro usuario
   [OK]     administradora acessa qualquer reuniao
   [OK]     usuario inativo nao acessa nada
   [OK]     usuario com e-mail invalido e rejeitado

=== Resultado: 29 verificacoes OK, 0 falha(s) ===
```

TesteDAO executa o CRUD completo no Oracle da FIAP: cria um usuário e uma reunião de teste, processa a reunião (análise e alertas gravados), lê pelos métodos de consulta, atualiza a transcrição, um alerta, o feedback e o usuário, e ao final exclui tudo o que criou, comprovando a exclusão em cascata. Saída da execução:

```
=== TOTVS Inteligencia Conversacional - Teste do CRUD (JDBC / Oracle) ===

Conexao: jdbc:oracle:thin:@//oracle.fiap.com.br:1521/orcl como RM561344
SELECT 1 FROM DUAL -> OK

-- CREATE
   usuario inserido: Usuario{Teste Java Sprint 3 <teste.java.1789351735500@company404.com.br> USER}
   transcricao inserida: ca99120f-35d4-4495-af47-858eafee4003 (PENDENTE)
   analise inserida: Analise{NEGATIVO | produtos=2 | oportunidades=0 | riscos=3 | scoreRisco=14,5}
   alertas inseridos: 5
     - [BAIXO] RISCO_INSATISFACAO - RISCO_INSATISFACAO (NOVO)
     - [MEDIO] RISCO_CONCORRENCIA - RISCO_CONCORRENCIA (NOVO)
     - [ALTO] RISCO_CHURN - RISCO_CHURN (NOVO)
     - [ALTO] CHURN_IMINENTE - Churn iminente (NOVO)
     - [MEDIO] CONCORRENTE_DETECTADO - CONCORRENTE_DETECTADO (NOVO)
   feedback inserido: Feedback{util: Risco de churn confirmado com o cliente.}

-- READ
   buscarPorId(transcricao): Transcricao{Clinica Vida Plena (teste Java) | Rafael Moura | PROCESSADA
| [LOCUTOR 2]: Estamos ha tres semanas com o chamado do RM abe...}
   buscarPorTranscricao(analise): Analise{NEGATIVO | produtos=2 | oportunidades=0 | riscos=3 |
scoreRisco=14,5}
     - RISCO INSATISFACAO: Insatisfacao relatada pelo cliente | Evidencia: "" | Severidade: BAIXO
     - RISCO CONCORRENCIA: Concorrente em negociacao com o cliente | Evidencia: "" | Severidade:
MEDIO | Concorrente: Senior
     - RISCO CHURN: Cliente verbalizou cancelamento ou troca de fornecedor | Evidencia: "" |
Severidade: ALTO
```

```
   listarPorCliente('Vida Plena'): 1 registro(s)
   buscarPorEmail(usuario): Teste Java Sprint 3
   listarPorTranscricao(feedback): 1 registro(s)
   total de transcricoes no banco: 624

-- UPDATE
   transcricao atualizada: Clinica Vida Plena (teste Java, atualizada) / NPS 9
   alerta atualizado: RESOLVIDO (esperado RESOLVIDO)
   feedback atualizado: Feedback{precisa ajuste: Severidade deveria ser MEDIO.}
   usuario atualizado: Teste Java Sprint 3 (atualizado)
   status final da transcricao: PROCESSADA (esperado PROCESSADA)

-- DELETE
   transcricao excluida; ainda existe? false
   analise ainda existe? false
   usuario excluido; ainda existe? false

=== CRUD concluido com sucesso ===
```

Os quatro testes JUnit 5 em `src/test/java` (churn com alerta iminente, ciclo de status, ciclo do alerta e extração de budget) também passam no `mvn package`.

6.6 Como executar

1. Requisitos: JDK 17 ou superior e acesso ao Oracle da FIAP. O driver `ojdbc11.jar` está em `lib/` e também é declarado no `pom.xml`.
2. Com Maven: `mvn package` compila, roda os testes JUnit e gera o jar. Sem Maven: compile com `javac -cp lib/ojdbc11.jar -d out` os arquivos de `src/main/java`.
3. Regras de negócio (sem banco): `java -cp target/classes com.tlean07.fiap.challenge.test.TesteRegrasNegocio`.
4. CRUD no Oracle: `java -cp "target/classes;lib/ojdbc11.jar" com.tlean07.fiap.challenge.test.TesteDAO`.
5. No IntelliJ ou Eclipse, abra a pasta como projeto Maven (ou adicione `lib/ojdbc11.jar` como biblioteca) e execute as classes do pacote `test`.

7. Documentos de referência

1. FIAP. Challenge 2026 – Engenharia de Software – TOTVS: Sprint 3 e Sprint 4. Orientações da disciplina Domain Driven Design – Java.
2. Grupo Company 404. Entrega de Domain Driven Design – Java – Sprints 1 e 2 (classes de domínio, herança e polimorfismo), junho de 2026.
3. Grupo Company 404. Entrega de Database Design – Sprint 3 (modelo relacional, DDL e DML), setembro de 2026.
4. Oracle. Oracle Database JDBC Developer's Guide, 21c; Oracle Database 19c SQL Language Reference.

Repositório da solução: github.com/TLean07/totvs-conversational-ai (branch oracle). Aplicação web: totvs-conversational-ai-oracle.vercel.app.